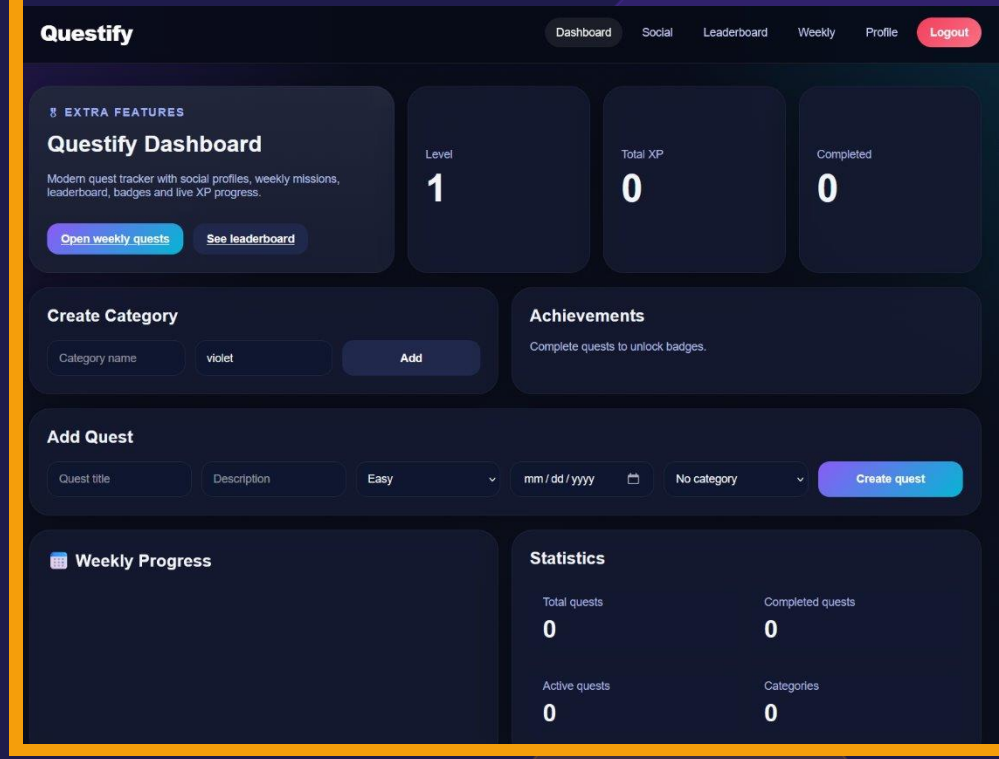
QUESTIFY

Gamified Task Management Web Application



Team Members

Bakyt Aiganym · 24B031680
Yessenbek Ainur · 24B031758

Angular 17+

Django REST

JWT Auth

WHAT WAS BUILT

App Overview

Questify turns everyday tasks into quests with difficulty levels, XP rewards, and a level-up progression system.

Users can manage personal quests, track weekly challenges, view their profile stats, and compete on a global leaderboard.

Pages & Navigation

- Dashboard – quest board + CRUD
- Profile – XP, level & badges
- Social – browse all users
- Leaderboard – XP rankings
- Weekly – weekly challenges
- Login / Register – JWT auth

JWT Auth

Login, register & secure routes

Quest CRUD

Create, update, delete quests

XP & Levels

Earn XP, level up on completion

Leaderboard

Global ranking by total XP

Weekly Goals

Auto-reset weekly challenges

Categories

Color-coded quest categories

Tech Stack: Angular 17+ · TypeScript · Django REST Framework · JWT (SimpleJWT) · SQLite · CORS

FRONT-END REQUIREMENTS

Angular 17+

Routing Module
7 named routes: /login, /register, /dashboard, /profile, /profile/:id, /social, /leaderboard, /weekly — with authGuard protection

✅ 7 routes (≥3 required)

JWT Authentication
auth.interceptor.ts attaches Bearer token to every request; authGuard blocks unauthenticated access; login & logout implemented

✅ Interceptor + Guard

Angular Services + HttpClient
AuthService & QuestService handle all API calls via HttpClient; no direct HTTP calls from components

✅ 2 services

Interfaces & Click Events
AuthInterface & QuestInterface defined; 4+ click events trigger API calls (add quest, complete quest, delete quest, logout, etc.)

✅ Interfaces + 4+ events

Forms with [(ngModel)]
Login form (username, password), Register form (username, email, password), Add Quest form (title, description, difficulty, category, deadline)

✅ 4+ form controls

@for / @if + Error Handling
@for to loop quests & leaderboard entries; @if for conditional UI (empty states, badges, auth check); errors shown inline to user

✅ Angular 17+ syntax + error display

BACK-END REQUIREMENTS

Django REST Framework

Models (4 total)

- User (Django built-in auth)
- Category – ForeignKey → User
- Quest – ForeignKey → User, Category
- QuestLog – ForeignKey → Quest, User

→ 4 FK relationships total

Serializers (4 total)

- serializers.Serializer: RegisterSerializer, LoginSerializer
- serializers.ModelSerializer: QuestModelSerializer, CategoryModelSerializer

Views

- FBV (2): register_view, login_view (@api_view + @permission_classes)
- CBV (2+): LogoutView, ProfileView, QuestListCreateView, CompleteQuestView…

JWT Authentication

Login & register endpoints return access + refresh tokens. LogoutView blacklists the refresh token via SimpleJWT.

Full CRUD – Quest Model

GET, POST /api/quests/quests/ · GET, PUT, DELETE /api/quests/quests/<id>/ · POST /api/quests/quests/<id>/complete/

Linked to Authenticated User

Every Quest and Category is saved with owner=request.user. QuestLog logs every action tied to the user.

CORS Configured

django-cors-headers installed in settings.py; CORS_ALLOW_ALL_ORIGINS = True for Angular dev server (localhost:4200)

Postman collection with all endpoints and example responses committed to the GitHub repository